

实 验 报 告

实验名称: 实验五 HMAX 模型实现

课程名称: 认知科学与类脑计算

实验地点: K2 227

实验日期: 2025/9/12

班 级: 23 级智能班

姓 名: 宋浩宇

学 号: 202300130183

一 实验目的：

加深对 HMAX 模型的理解，能够使用 HMAX 模型解决简单问题

二 实验环境：

主系统：Windows 11 家庭中文版 23H2 ; ArchLinux

编辑器：Visual Studio Code ; PyCharm 2025.1.2 ; neovim 0.11

Python 解释器：Python 3.9

Pytorch 版本：2.8.0

CUDA 版本：12.8.97

硬件环境：

CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz

内存：32.0 GB (31.6 GB 可用)

磁盘驱动器：NVMe WD_BLACKSN850X2000GB

显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU

CUDA 核心数：7424

三 实验内容：

根据 HMAX 模型的相关知识，使用 Python 语言实现一个简单的 HMAX 模型。

四 实验步骤：

1. 实验程序：

```
import os
import pickle
import numpy as np
from tqdm import tqdm
from scipy.ndimage import convolve
from scipy.signal import correlate
from skimage.measure import block_reduce
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from joblib import dump
from torchvision import datasets, transforms

MNIST_PATH = r'F:\DATASET\MNIST\mnist_dataset_dir'
IMG_SIZE = (64, 64)
scales = [2, 4, 6, 8]
orientations = [0, np.pi/4, np.pi/2, 3*np.pi/4]
gamma = 0.5
sigma_ratio = 0.56
```

```

pool_size = 4
patch_size = 4
num_patches = 200
patch_from_images = 2000
train_samples = 3000
test_samples = 1000
cache_dir = "./hmax_cache"
os.makedirs(cache_dir, exist_ok=True)
def gabor_kernel(theta, lambd, sigma=None, gamma=gamma):
    if sigma is None:
        sigma = sigma_ratio * lambd
    size = int(3 * sigma)
    x, y = np.meshgrid(np.arange(-size, size+1), np.arange(-size,
size+1))
    x0 = x * np.cos(theta) + y * np.sin(theta)
    y0 = -x * np.sin(theta) + y * np.cos(theta)
    gb = np.exp(-(x0**2 + (gamma**2) * y0**2) / (2 * sigma**2)) *
np.cos(2 * np.pi * x0 / lambd)
    return gb
def s1_layer(image, scales=scales, orientations=orientations):
    H, W = image.shape
    s1 = np.zeros((len(scales), len(orientations), H, W),
dtype=np.float32)
    for si, lambd in enumerate(scales):
        sigma = sigma_ratio * lambd
        for oi, theta in enumerate(orientations):
            kern = gabor_kernel(theta, lambd, sigma=sigma)
            s1[si, oi] = convolve(image, kern, mode='reflect')
    return s1
def c1_layer(s1_maps, pool_size=pool_size):
    num_scales, num_orients, H, W = s1_maps.shape
    pooled = np.zeros((num_scales, num_orients, H // pool_size, W //
pool_size), dtype=np.float32)
    for si in range(num_scales):
        for oi in range(num_orients):
            pooled[si, oi] = block_reduce(s1_maps[si, oi],
block_size=(pool_size, pool_size), func=np.max)
    bands = []
    si = 0
    while si < num_scales:
        if si + 1 < num_scales:
            bands.append(np.maximum(pooled[si], pooled[si + 1]))
            si += 2
        else:

```

```

        bands.append(pooled[si])
        si += 1
    return np.stack(bands, axis=0)
def compute_c2_for_one_c1(c1_maps, patches):
    num_bands, num_orients, Hc, Wc = c1_maps.shape
    K = patches.shape[0]
    c2 = np.zeros((K,), dtype=np.float32)
    for k in range(K):
        patch = patches[k]
        patch_energy = np.sum(patch**2)
        best_sim = -np.inf
        for b in range(num_bands):
            num_map = np.zeros((Hc, Wc), dtype=np.float32)
            denom_map = np.zeros((Hc, Wc), dtype=np.float32)
            for o in range(num_orients):
                feat = c1_maps[b, o]
                num_map += correlate(feat, patch[o], mode='same')
                denom_map += correlate(feat**2, np.ones((patch_size,
patch_size)), mode='same')
            denom = np.sqrt(denom_map * (patch_energy + 1e-12)) + 1e-12
            sim_map = num_map / denom
            max_sim = np.max(sim_map)
            if max_sim > best_sim:
                best_sim = max_sim
        c2[k] = best_sim
    return c2
def extract_features(dataset, num_samples, patches=None,
cache_name=None):
    feats, labels = [], []
    for i in tqdm(range(num_samples), desc=f"提取特征"):
        img, label = dataset[i]
        img = img.squeeze().numpy()
        s1 = s1_layer(img)
        c1 = c1_layer(s1)
        c2 = compute_c2_for_one_c1(c1, patches)
        feats.append(c2)
        labels.append(label)
    feats = np.stack(feats, axis=0)
    labels = np.array(labels, dtype=np.int64)
    if cache_name:
        with open(cache_name, 'wb') as f:
            pickle.dump((feats, labels), f)
    return feats, labels

```

```

patch_cache_file = os.path.join(cache_dir,
f"patches_{num_patches}.pkl")
if os.path.exists(patch_cache_file):
    print("正在加载缓存的补丁...")
    with open(patch_cache_file, 'rb') as f:
        patches = pickle.load(f)
else:
    transform = transforms.Compose([transforms.ToTensor(),
transforms.Resize(IMG_SIZE)])
    train_dataset = datasets.MNIST(root=MNIST_PATH, train=True,
download=True, transform=transform)
    print("正在随机提取补丁...")
    sampled = []
    for i in tqdm(range(patch_from_images), desc="生成补丁"):
        img, _ = train_dataset[i]
        img = img.squeeze().numpy()
        s1 = s1_layer(img)
        c1 = c1_layer(s1)
        b = np.random.randint(0, c1.shape[0])
        Hc, Wc = c1.shape[2], c1.shape[3]
        if Hc - patch_size + 1 <= 0 or Wc - patch_size + 1 <= 0:
            continue
        h = np.random.randint(0, Hc - patch_size + 1)
        w = np.random.randint(0, Wc - patch_size + 1)
        sampled.append(c1[b, :, h:h+patch_size, w:w+patch_size])
        if len(sampled) >= num_patches:
            break
    patches = np.stack(sampled, axis=0)
    with open(patch_cache_file, 'wb') as f:
        pickle.dump(patches, f)
    print("补丁已保存。")
transform = transforms.Compose([transforms.ToTensor(),
transforms.Resize(IMG_SIZE)])
train_dataset = datasets.MNIST(root=MNIST_PATH, train=True,
download=True, transform=transform)
test_dataset = datasets.MNIST(root=MNIST_PATH, train=False,
download=True, transform=transform)
train_cache = os.path.join(cache_dir, f"train_{train_samples}.pkl")
test_cache = os.path.join(cache_dir, f"test_{test_samples}.pkl")
if os.path.exists(train_cache):
    print("加载训练集特征...")
    with open(train_cache, 'rb') as f:
        train_features, train_labels = pickle.load(f)
else:

```

```

    print("正在提取训练集特征...")
    train_features, train_labels = extract_features(train_dataset,
train_samples, patches, train_cache)
if os.path.exists(test_cache):
    print("加载测试集特征...")
    with open(test_cache, 'rb') as f:
        test_features, test_labels = pickle.load(f)
else:
    print("正在提取测试集特征...")
    test_features, test_labels = extract_features(test_dataset,
test_samples, patches, test_cache)
print("正在训练 SVM 分类器...")
scaler = StandardScaler()
trainF = scaler.fit_transform(train_features)
testF = scaler.transform(test_features)
svc = SVC(kernel='linear')
svc.fit(trainF, train_labels)
dump({'svc': svc, 'scaler': scaler, 'patches': patches},
os.path.join(cache_dir, "model.joblib"))
pred = svc.predict(testF)
acc = accuracy_score(test_labels, pred)
print(f"测试集准确率: {acc:.4f}")

```

2. 程序实验结果:

```

正在提取测试集特征...
特征: 100%|██████████| 1000/1000 [03:46<00:00, 4.42it/s]
正在训练 SVM 分类器...
测试集准确率: 0.8790

```

3. 分析和改进:

当前的代码因为计算方式很原始没有做优化,算的太慢,光是算一个 `patches` 就得十多分钟,提取特征又是十多分钟,等他出结果的过程太折磨了,改进的话就是用 `numba` 编译一部分字节码,或者给 `compute_c2_for_one_c1` 加一个并行计算。而且结果不是特别好,还需要调参。

五 实验小结:

从结果上来看,模型的准确度还算达标,能够成功解决这个问题。但是当前代码计算速度太慢,还有优化空间。